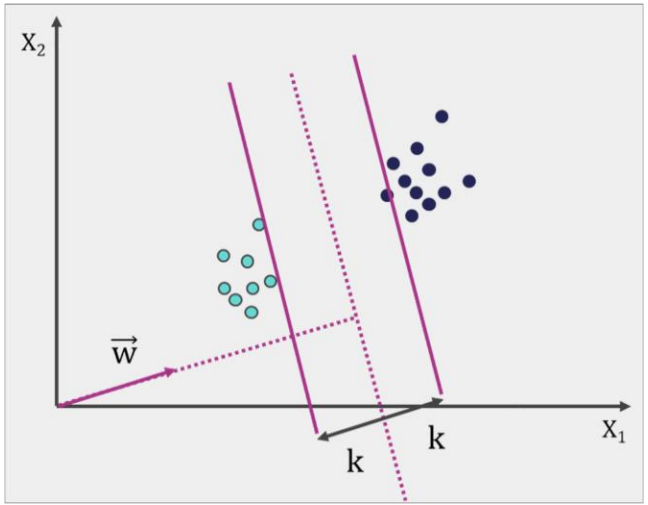
QpiAI™ - Joint AI Training and Certification Program

Support Vector Machines

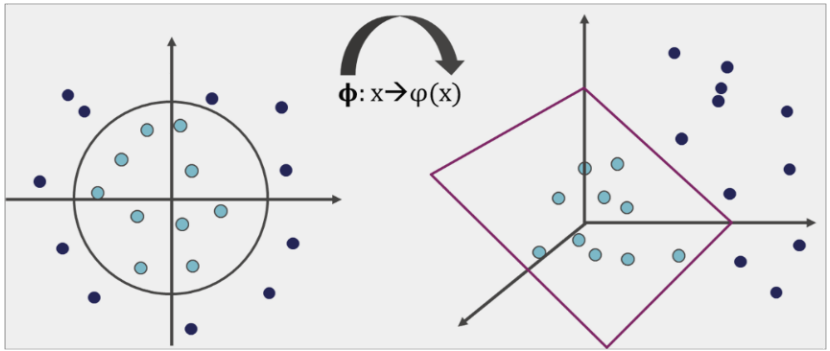
Arpit Jain, PhD

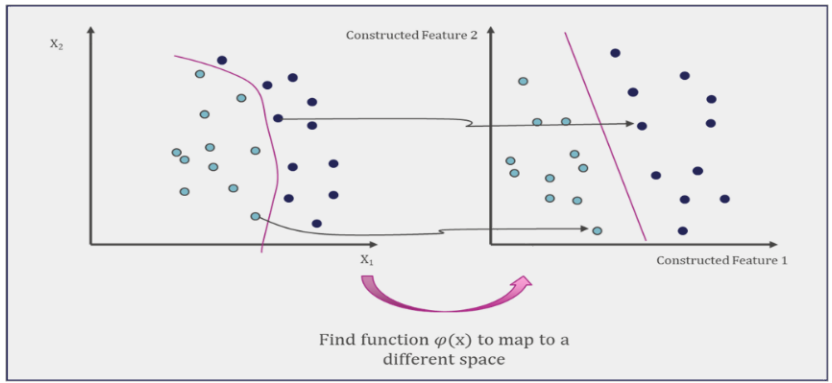
Support Vector Machines?

Linearly Separable



Non-Linearly Separable





Prerequisite for SVM:

Length of a vector

The length of a vector x is called its norm, which is written as $||x||$. The Euclidean norm formula to calculate the Norm of a vector $\mathbf{x}= (x_1, x_2, \dots x_n)$ is:

$$||x|| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

Direction of a vector

The direction of a vector $\mathbf{x}=(x_1, x_2)$ is written as $\vec{w}$**,** and is defined as:

$$\vec{w} = \left(\frac{x_1}{||x||}, \frac{x_2}{||x||}\right)$$

What is a Hyperplane

In a p-dimensional space a hyperplane is a flat affine subspace of dimension p-1.

For example in 2-dimension a hyperplane is a 1-dimension subspace which a line. Equation of line in given by

$$y = ax+b$$

Now replace x with x_1 and y with x_2 , now equation will look something like this

$$ax_1-x_2+b = 0$$

Again if x = (x_1,x_2) and w = (a,-1) the above equation can we written as

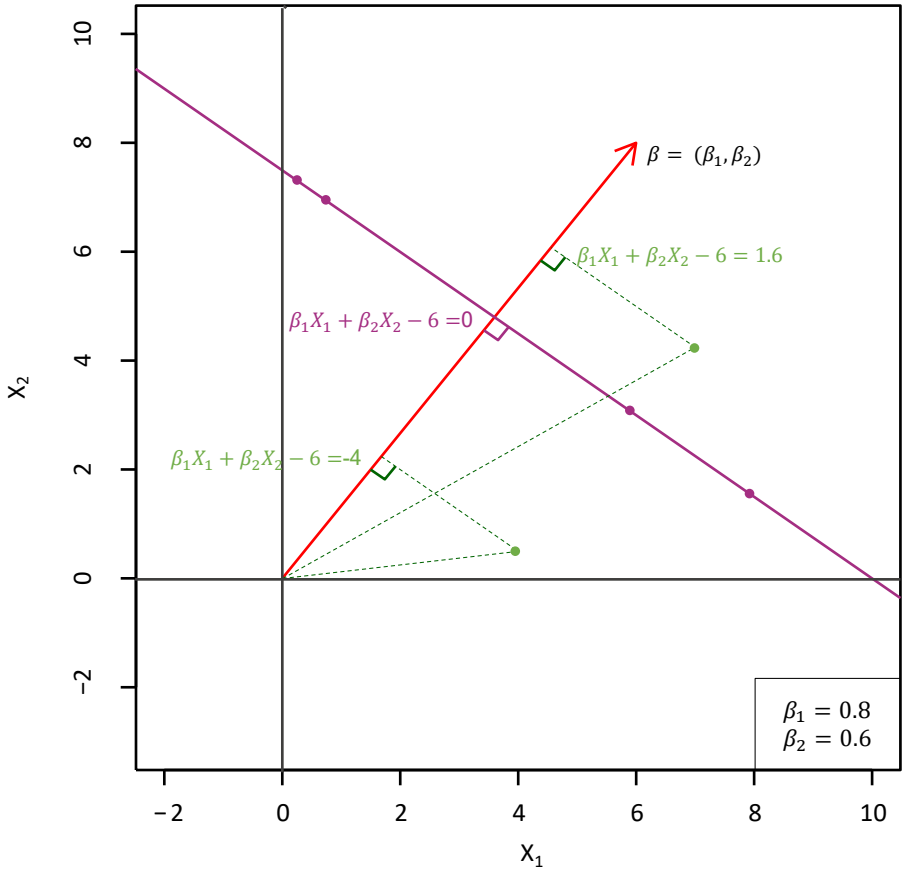
$$\mathbf{w}.\mathbf{x}+b = 0$$

The general equation of a hyperplane can be written as:

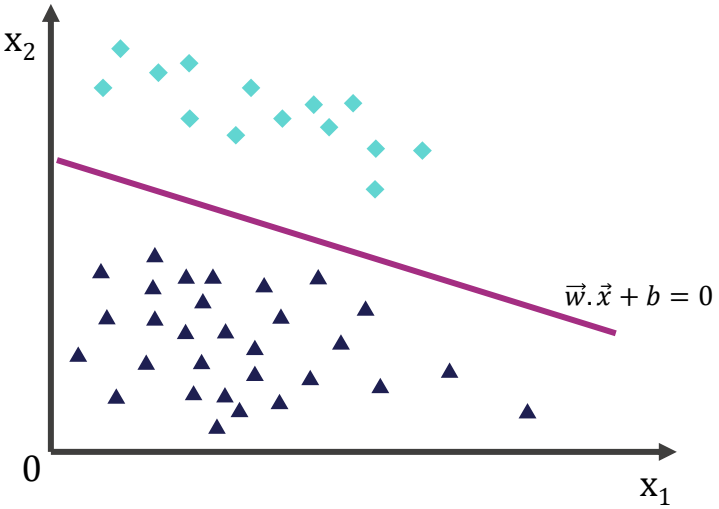
$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p = 0$$

The vector $\beta = (\beta_0, \beta_1, \beta_2 \ldots \beta_p)$ points in orthogonal direction to the surface of a hyperplane and hence is called the normal vector.

Hyperplane in 2 Dimensions



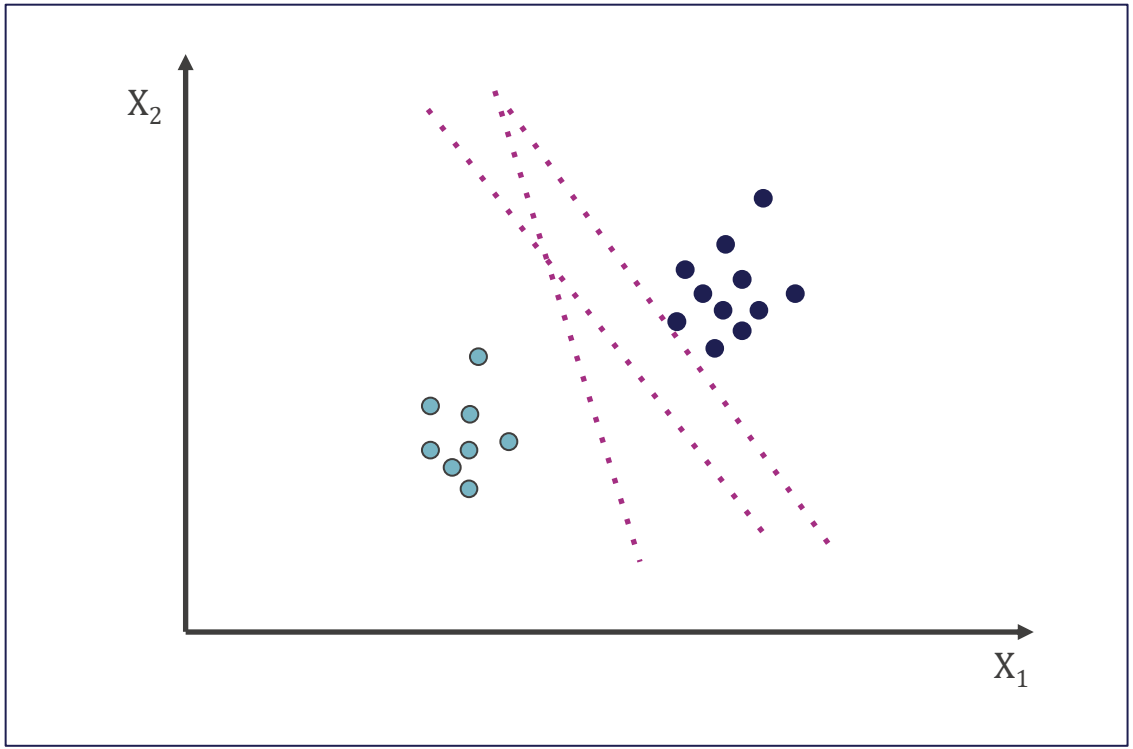
Linear separating through a hyperplane



With each vector x there is a label y, these labels can have value of +1 and -1. The points that lie on the line have a label of 0. The points lying above the line have a label of +1 and the once lying below the line are labelled as -1.

$$h(x_i) = \begin{cases} +1, & \text{if } \vec{w}.\vec{x} + b \geq 0 \\ -1, & \text{if } \vec{w}.\vec{x} + b < 0 \end{cases}$$

Which Separating Hyperplane to use?



Support Vector Machines

Three Main ideas

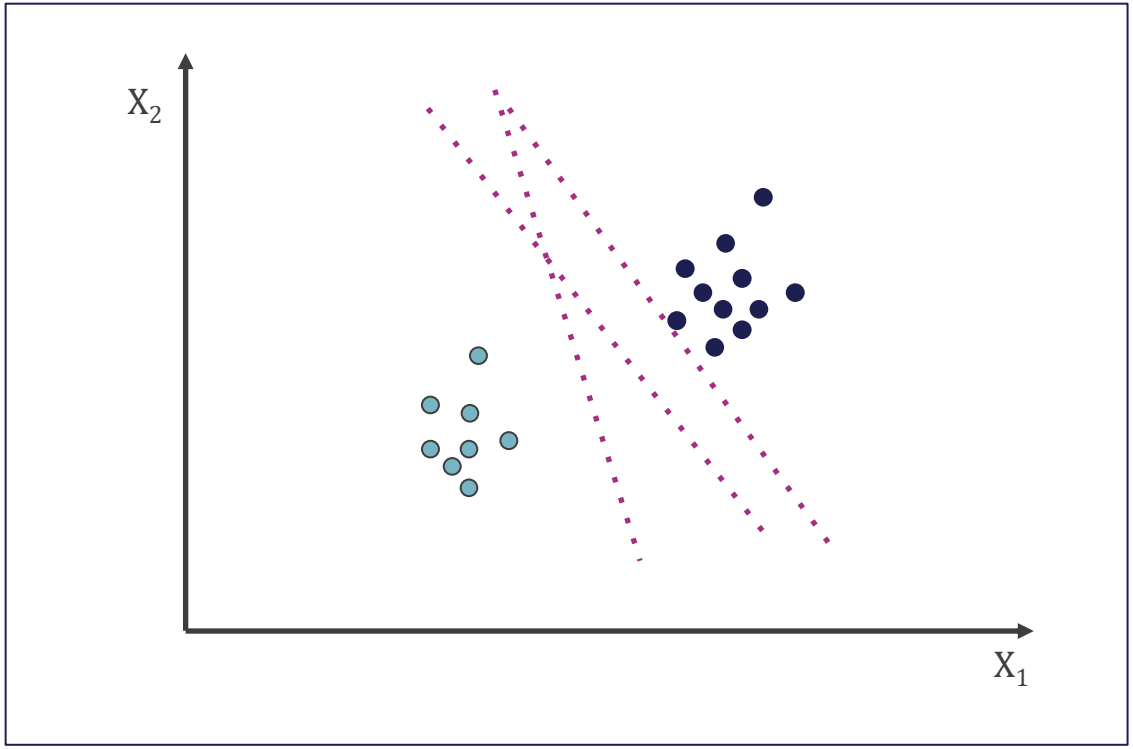
1. Maximal Margin Classifier(Linear boundary): **Maximize margin**
2. Support Vector Classifier(Non-Separable Case): **Have a penalty term for misclassifications**
3. Support Vector Machine (Non-Linear boundaries): **Reformulate Problem so that data are mapped implicitly to higher dimension**

Support Vector Machines

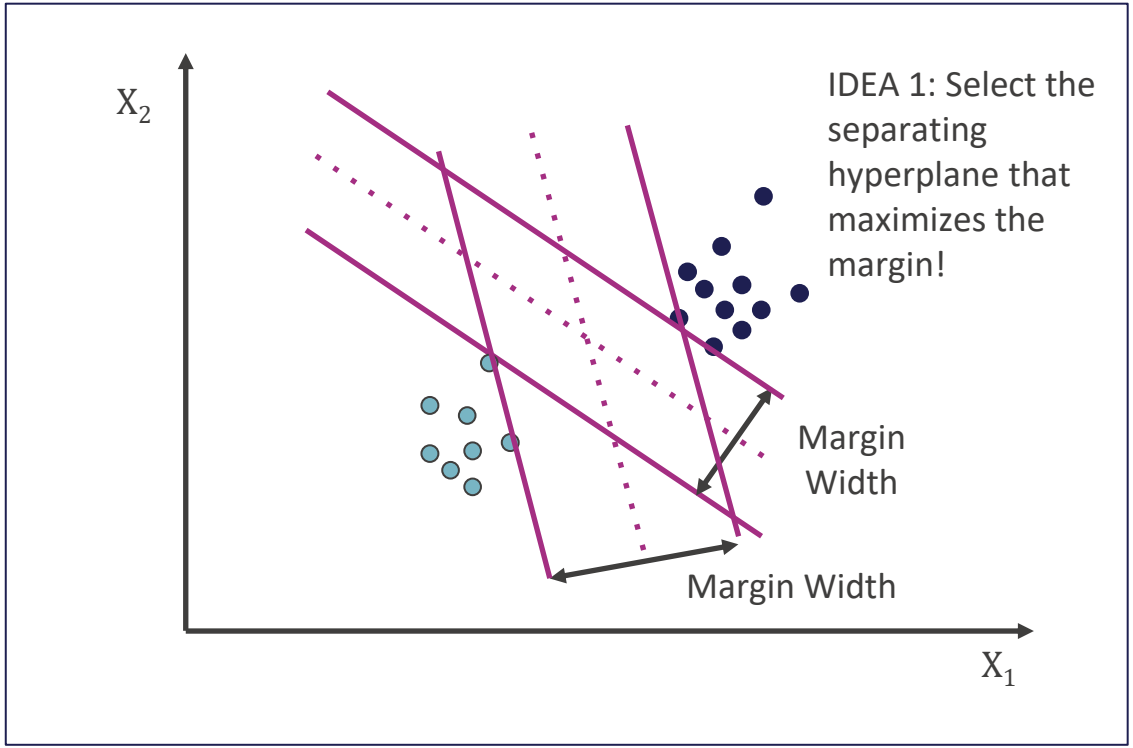
Three Main ideas

1. Maximal Margin Classifier(Linear boundary): **Maximize margin**
2. Support Vector Classifier(Non-Separable Case): **Have a penalty term for misclassifications**
3. Support Vector Machine (Non-Linear boundaries): **Reformulate Problem so that data are mapped implicitly to higher dimension**

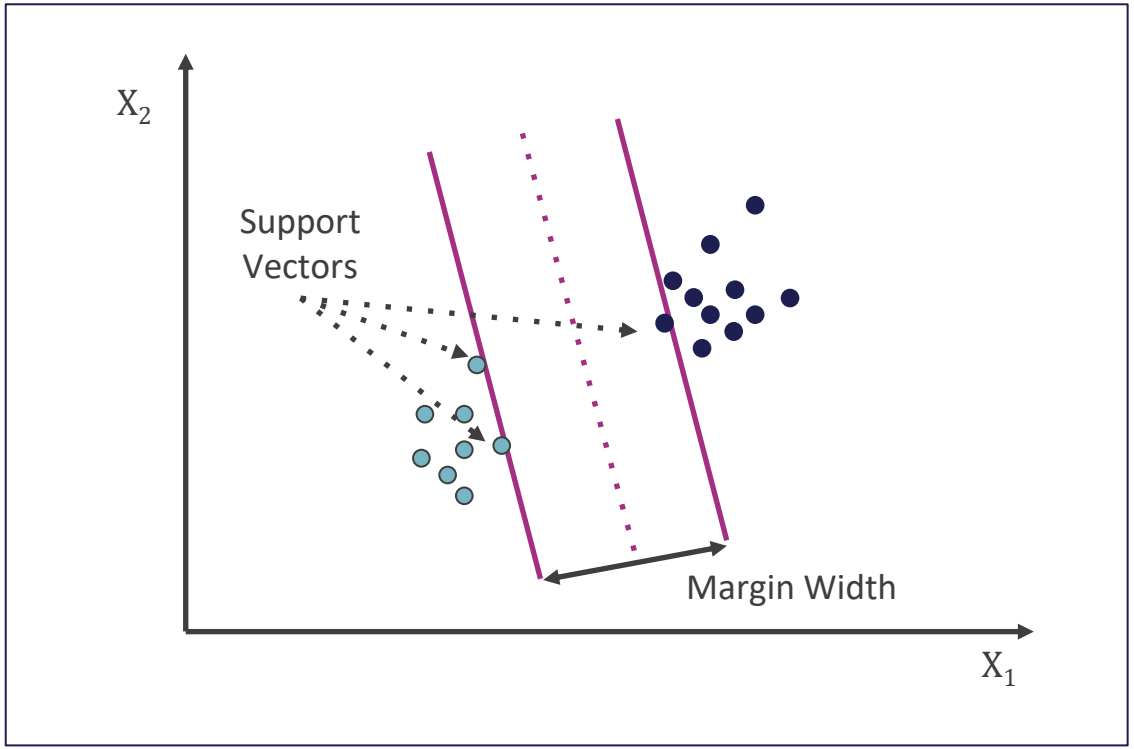
Which Separating Hyperplane to use?



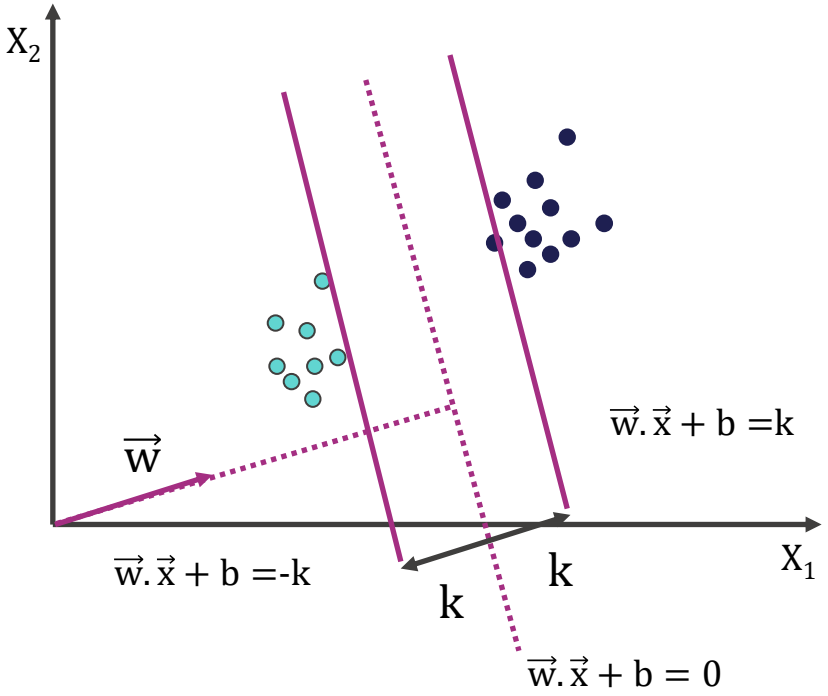
Maximizing the Margin



Support Vector



Setting Up the Optimization Problem



The width of the margin is:

$$\frac{2|k|}{||\vec{w}||}$$

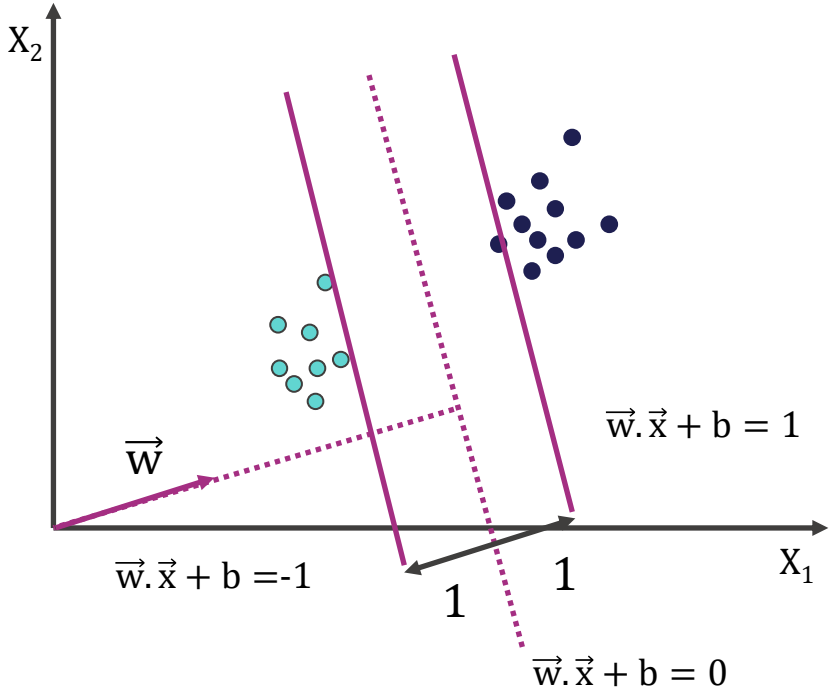
So, the problem is:

$$\max \frac{2|k|}{||\vec{w}||}$$

s. t. $(\vec{w}.\vec{x} + b) \geq k, \forall x$ of class 1

$(\vec{w}.\vec{x} + b) \leq -k, \forall x$ of class 2

Setting Up the Optimization Problem



Scaling w, b so that k=1, the problem becomes:

$$\max \frac{2}{||\vec{w}||}$$

$$s.t.\ (\vec{w}.\vec{x} + b) \geq 1, \forall x \text{ of class } 1$$

$$(\vec{w}.\vec{x} + b) \leq -1, \forall x \text{ of class } 2$$

Setting Up the Optimization Problem

If class 1 corresponds to 1 and class 2 corresponds to -1, we can rewrite

$$(\vec{w}.x_i + b) \geq 1, \forall x_i \text{ with } y_i = 1$$

$$(\vec{w}.x_i + b) \leq -1, \forall x_i \text{ with } y_i = -1$$

as

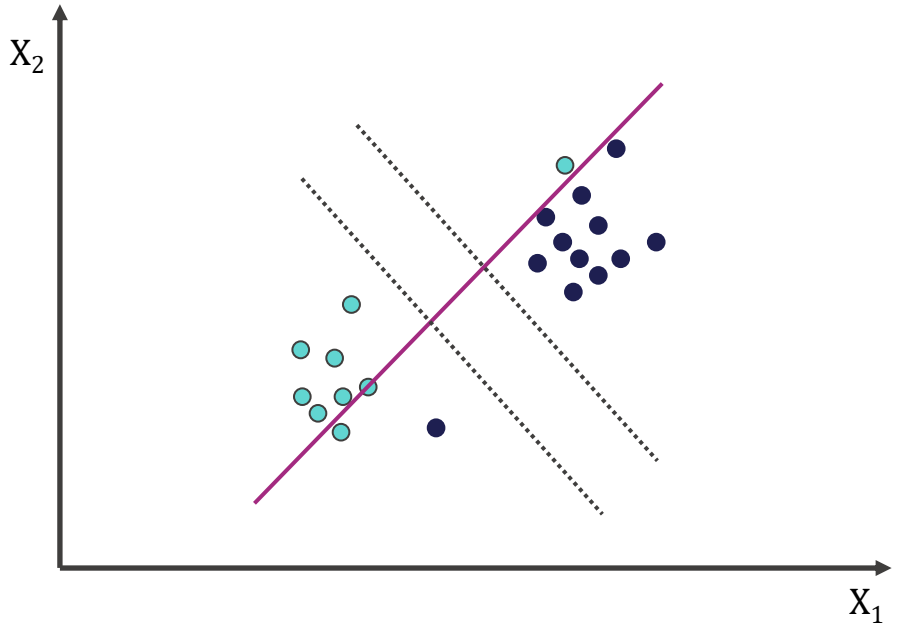
$$y_i (\vec{w}.x_i + b) \geq 1, \forall x_i$$

So the problem becomes

$$\max \frac{2}{||\vec{w}||} \quad \text{or} \quad \min \frac{1}{2} ||\vec{w}||^2$$

$$\text{s.t. } y_i(\vec{w}.x_i + b) \geq 1, \forall x_i \qquad \text{s.t. } y_i(\vec{w}.xi + b) \geq 1, \forall x_i$$

Non-Separable Data

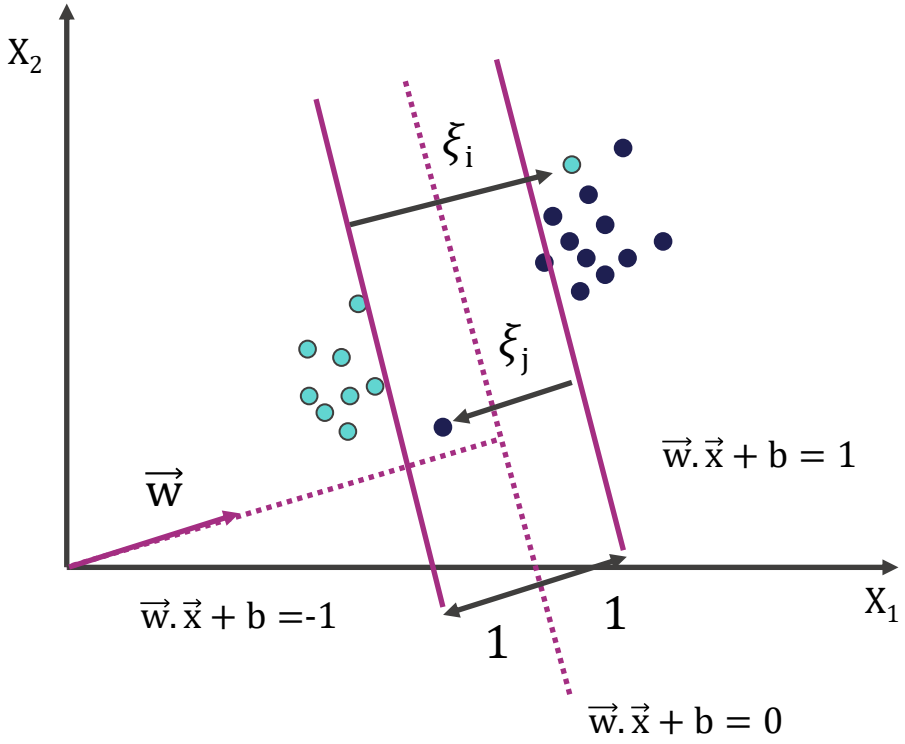


Support Vector Machines

Three Main ideas

1. Maximal Margin Classifier(Linear boundary): **Maximize margin**
2. Support Vector Classifier (Non-Separable Case): **Have a penalty term for misclassifications**
3. Support Vector Machine (Non-Linear Boundaries): **Reformulate Problem so that data are mapped implicitly to higher dimension**

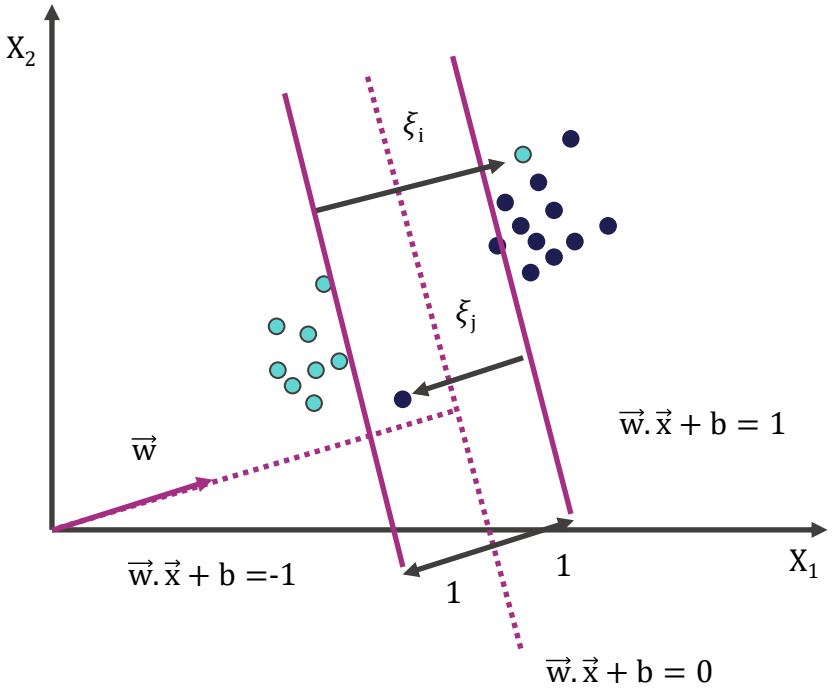
Non-Separable Data



Introduce slack variables ξ_i

Allow some instance to fall within the margin but penalize them

Formulating the optimization Problem



Constraint becomes:

$$y_i(\vec{w}.\vec{x}_i + b) \geq 1 - \xi_i, \forall x_i$$

$$\xi_i \geq 0$$

Objective function penalizes for misclassified instance and those within the margin

$$\min \frac{1}{2}||\vec{w}||^2 + C\sum_i \xi_i$$

C trades-off margin width & misclassifications

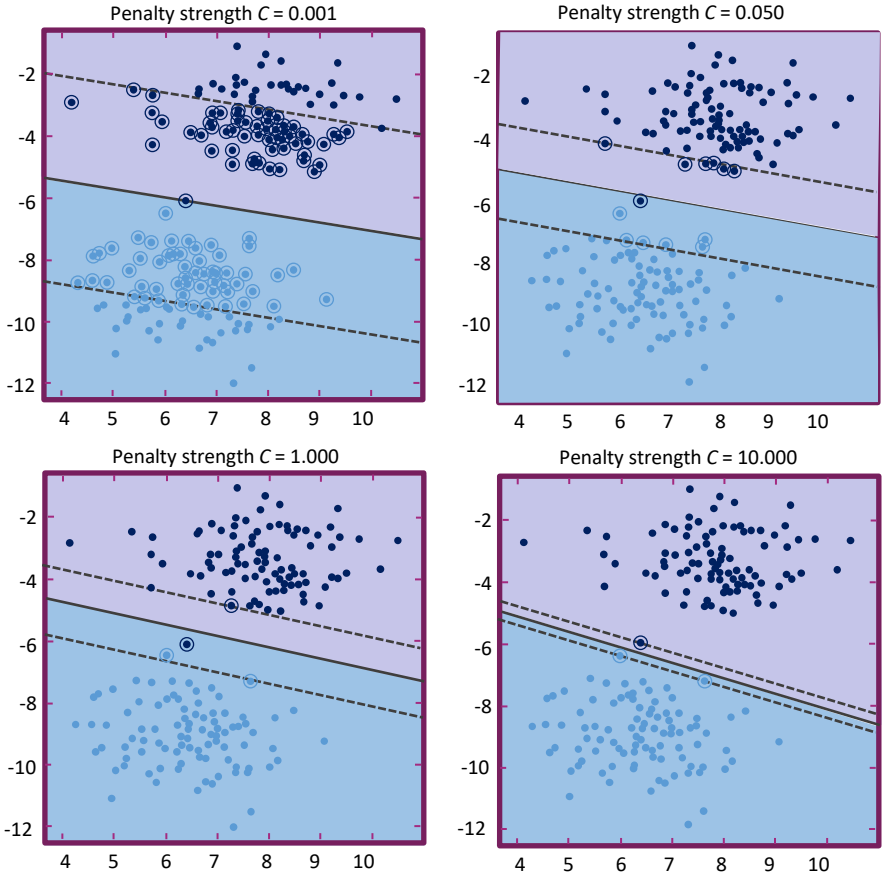
Linear, Soft-Margin SVMs

$$\min \frac{1}{2}\left|\left|\vec{w}\right|\right|^2 + C\sum_i \xi_i \qquad y_i(\vec{w}.\vec{x}_i + b) \geq 1 - \xi_i, \forall xi$$

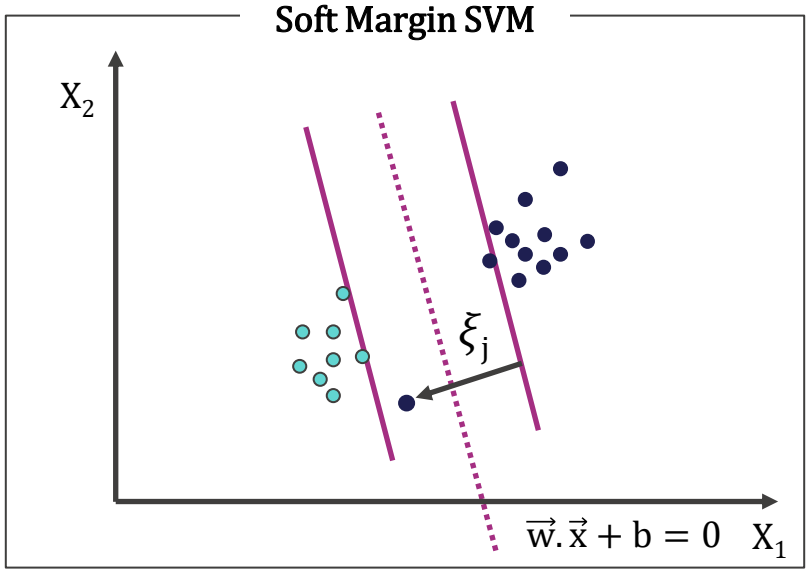
$$\xi_i \geq 0$$

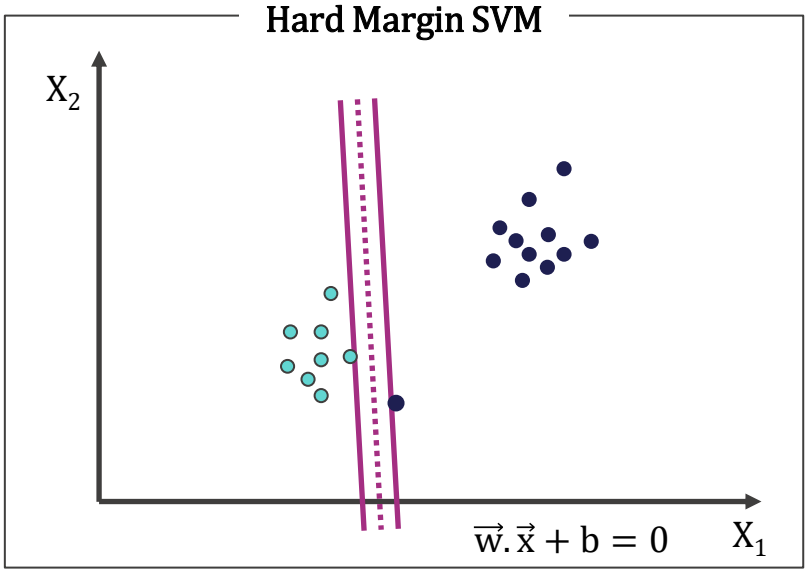
- Algorithm tries to keep ξ_i at zero while maximizing margin
- C: penalty for misclassification
- As C→∞, we get closer to the hard-margin solution

Trade-off of Penalty(C)



Robustness of soft vs Hard Margin SVMs



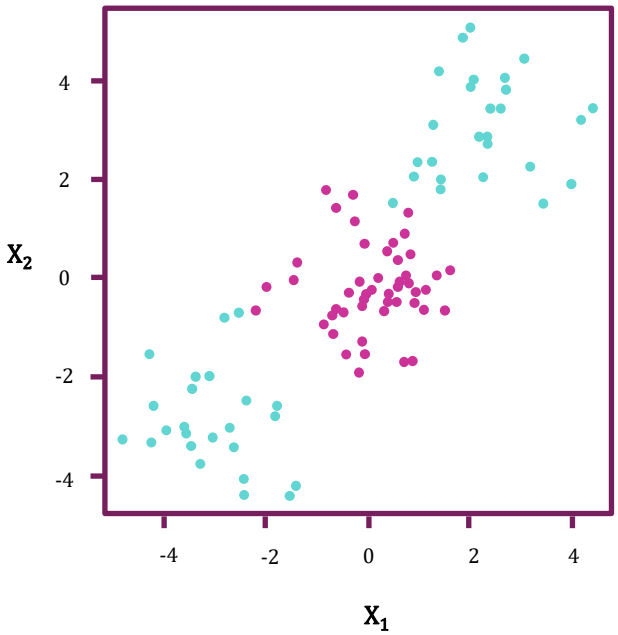


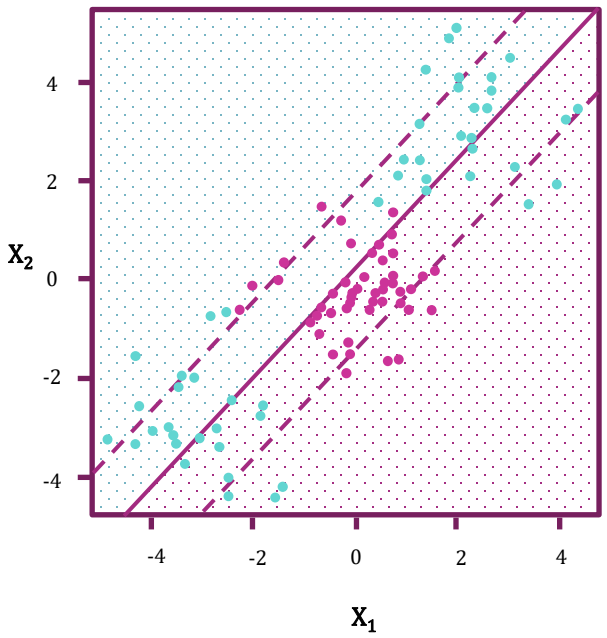
Support Vector Machines

Three Main ideas

1. Maximal Margin Classifier(Linear boundary): **Maximize margin**
2. Support Vector Classifier(Non-Separable Case): **Have a penalty term for misclassifications**
3. Support Vector Machine (Non-Linear boundaries): **Reformulate Problem so that data are mapped implicitly to higher dimension**

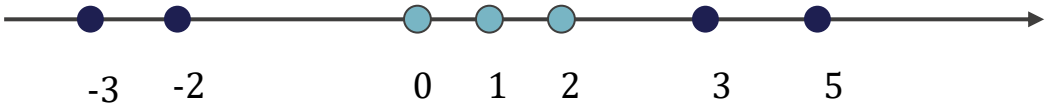
Non-Linear Data



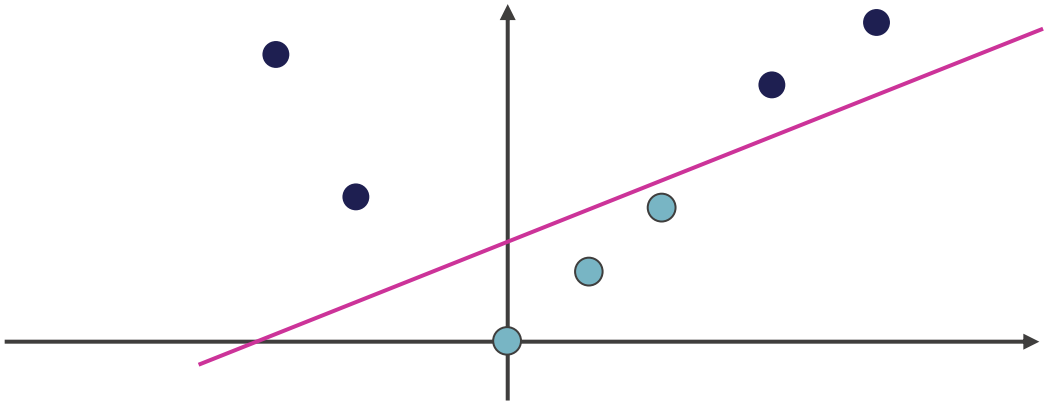


Non-Linear Mapping

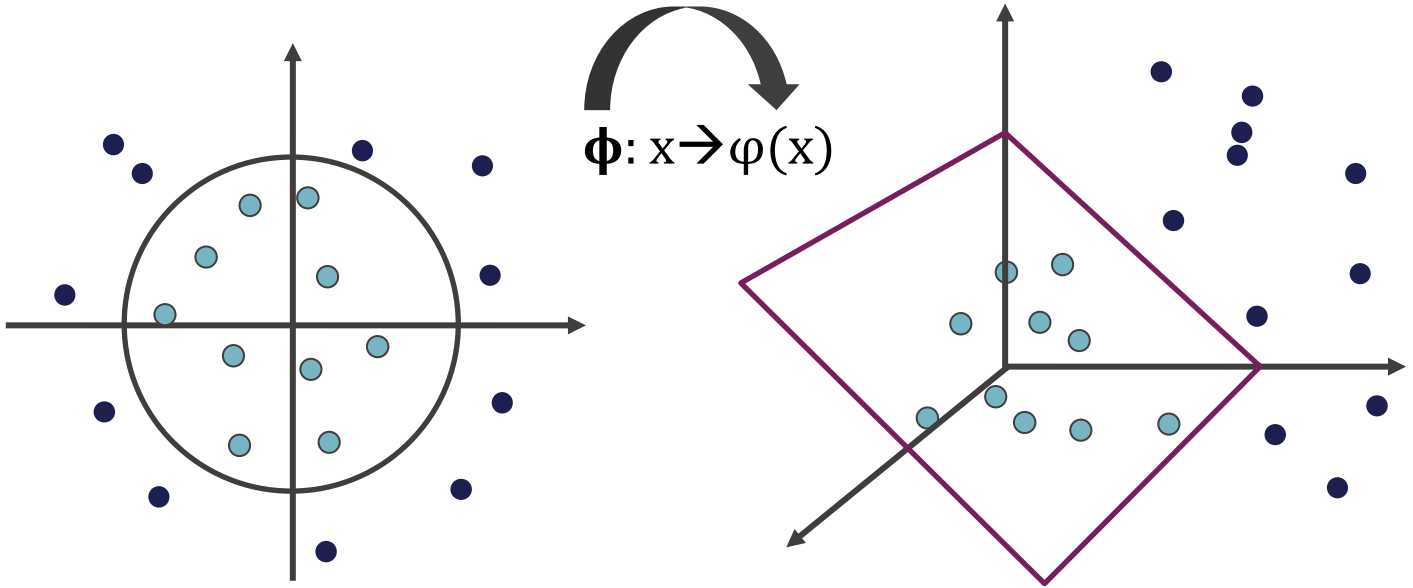
One dimensional space, not linearly separable



Lift to two-dimensional space with $\phi(x)=(x,x^2)$



Linear Classifiers in High-Dimensional Spaces



Mapping Data to a high-Dimensional Space

- Find function φ(x) to map to a different space, them SVM formulation becomes:

$$\min \frac{1}{2}\left|\left|\vec{w}\right|\right|^2 + C\sum_i \xi_i \qquad \text{s.t. } y_i(\vec{w}.\,\varphi(xi) + b) \geq 1 - \xi_i, \forall x_i$$

$$\xi_i \geq 0$$

- Data appear as φ(x), weights w are now weights in the new space
- Explicit mapping expensive if φ(x) is very high dimensional
- Can we solve the problem without explicitly mapping the data?

The Kernel Trick:

It allows us to operate in the original feature space without computing the coordinates of the data in a higher dimensional space

Let's look at an example

$$x=(x_1,x_2,x_3)^T$$

$$y=(y_1,y_2,y_3)^T$$

Here x and y are two data points in 3 dimensions. Let's assume that we need to map x and y to 9-dimensional Space. We need to do following calculations to get the final result, Which is just a scalar. The computational complexity, in this case, is $O(n^2)$

$$\Phi(x)=(x_1^2, x_1x_2,x_1x_3,x_2x_1, x_2^2,x_2x_3,x_3x_1,x_3x_2, x_3^2)^T$$

$$\Phi(y)=(y_1^2, y_1y_2,y_1y_3,y_2y_1, y_2^2,y_2y_3,y_3y_1,y_3y_2, y_3^2)^T$$

$$\Phi(x)^T \Phi(y)= \sum_{i,j=1}^{3} x_i x_j y_i y_j$$

Feature Expansion

- Enlarge the space of features by including transformations; e.g $X_1^2, X_1^3, X_1X_2, X_1X_2^2, \ldots$. Hence go from a p-dimensional space to a n > p dimensional space.
- Fit a support-vector classifier in the enlarged space.
- This results in non-linear decision boundaries in the original space.

Example: Suppose we use $(X_1, X_2, X_1^2, X_2^2, X_1X_2)$ instead of just (X_1, X_2). Then the decision boundary would be of the form

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 = 0$$

This leads to nonlinear decision boundaries in the original space (quadratic conic sections).

Before we proceed further let us understand the role of inner products in the support vector classifiers.

Inner products and support vectors

$$\langle x_i, x_i' \rangle = \sum_{j=1}^{p} x_{ij} x_{i'j}$$

Inner product between vectors

- Using the function above the linear support vector classifier can be represented as:

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i \langle x, x_i \rangle$$

- n parameters

- To estimate the parameters $\alpha_1, \ldots, \alpha_n$ and β_0, all we need are the $\binom{n}{2}$ inner products $\langle x_i, x_i' \rangle$ between all pairs of training observations.

It turns out that most of the $\hat{\alpha}_i$ can be zero:

$$f(x) = \beta_0 + \sum_{i \epsilon S} \hat{\alpha}_i \langle x, x_i \rangle$$

S is the support set of indices i such that $\hat{\alpha}_i > 0$

Kernels and Support Vector Machines

- If we can compute inner-products between observations, we can fit a SV classifier. Can be quite abstract!
- Some special kernel functions can do this for us. E.g.

$$K(x_i, x_i') = \left(1 + \sum_{j=1}^{p} x_{ij} x_i{}'_j\right)^d$$

computes the inner-products needed for d dimensional polynomials $\binom{p+d}{d}$ basis functions!

Try it for $p = 2$ and $d = 2$

- The solution has the form

$$f(x) = \beta_0 + \sum_{i \epsilon S} \hat{\alpha}_i K\langle x, x_i \rangle$$

Different kernel functions

There are different kernels. The most popular ones are the polynomial kernel and the radial basis function (RBF) kernel

1. Polynomial kernel:

$$K(x, y)=(x^T y+1)^d$$

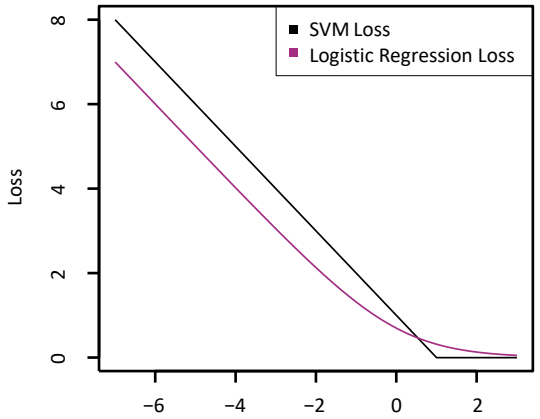
2. Radial basis function (RBF) kernel

$$K(x, y)=e^{-\gamma||x-y||^2}, \gamma > 0$$

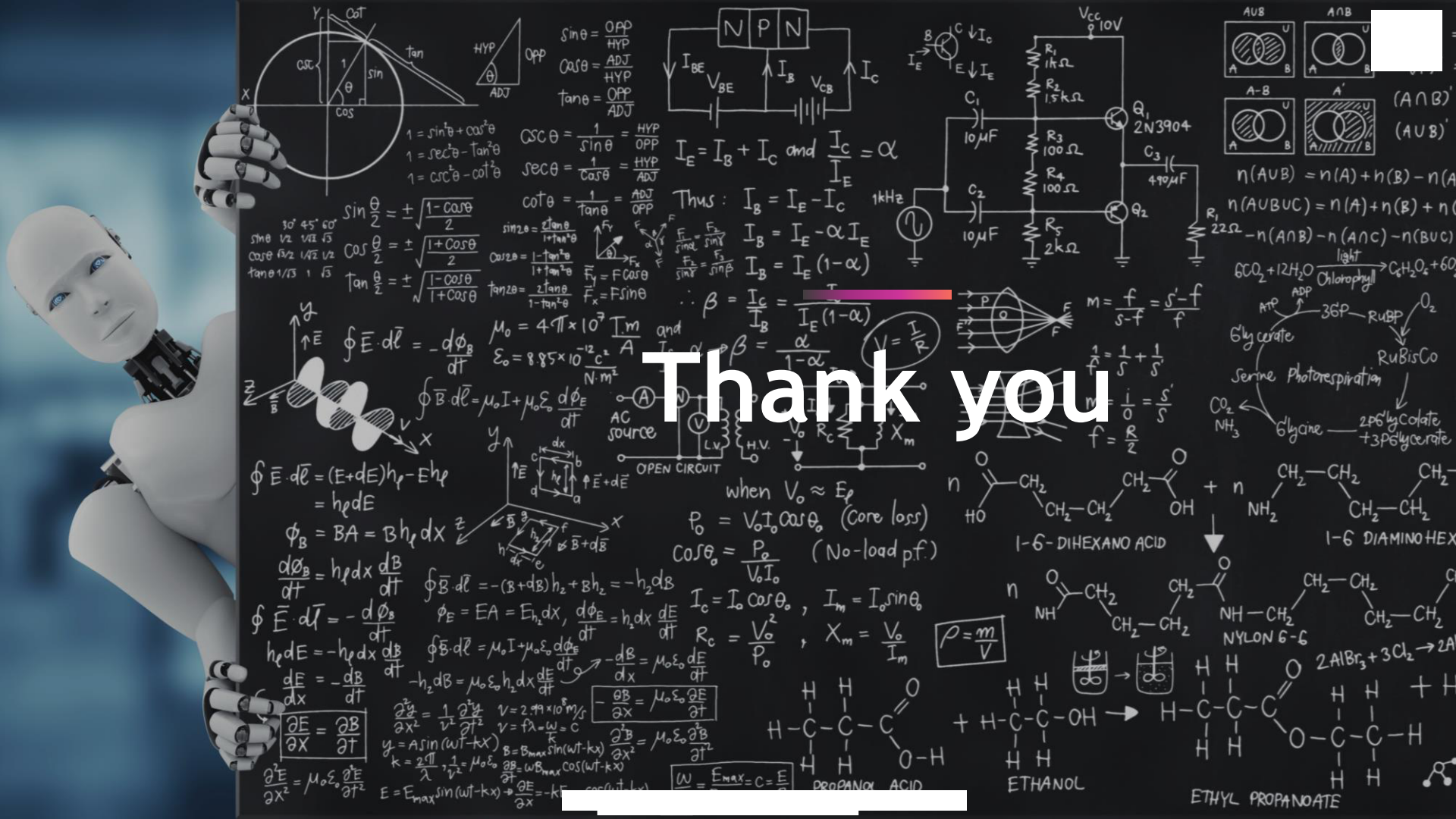
Support Vector versus Logistic Regression?

With $f(X) = \beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p$ can rephrase support-vector classifier optimization as

$$\underset{\beta_0, \beta_1, \ldots, \beta_p}{\text{minimize}} \left\{ \sum_{i=1}^{n} \max[0, 1 - y_i f(x_i)] + \lambda \sum_{j=1}^{p} \beta_j^2 \right\}$$



This has the form loss plus penalty.
The loss is known as the hinge loss.
Very similar to "loss" in logistic regression (negative log-likelihood).


Thank you